SISTEMAS DISTRIBUÍDOS | TEMA 7 — AMAZON ECS

Orquestração de containers com Amazon ECS

Relatório técnico parcial: arquitetura, implementação, método e resultados de validação local

Disciplina	Equipe	Prazo final	Data deste relatório
Sistemas Distribuídos	4 integrantes	01/12/2026	14/11/2026

Resumo

Este relatório descreve o projeto do Tema 7: uma API HTTP containerizada, implantada como serviço replicado no Amazon ECS com Fargate, para demonstrar replicação, substituição automática de tasks após falha e escalabilidade automática por uso de CPU. São apresentados a fundamentação, a arquitetura, a implementação de cada frente da equipe, o método dos quatro experimentos (baseline, scale-out, scale-in e falha) e os resultados disponíveis até 14/11/2026. Nessa data, a aplicação, a infraestrutura como código, o Auto Scaling e as ferramentas de medição estavam prontos e validados localmente, mas nenhum recurso havia sido criado na AWS, por pendências de conta e orçamento. Por isso, todos os números aqui são de ambiente local e estão identificados como tal.

Regra de leitura. Nenhum número deste relatório foi estimado ou inventado. Cada valor indica o arquivo do repositório de onde foi tirado. Resultados locais (Docker com 0,25 vCPU) validam a aplicação e as ferramentas; não comprovam o comportamento no Fargate.

1. Introdução

Aplicações distribuídas precisam continuar disponíveis quando uma instância falha e acompanhar a variação de carga sem intervenção manual. Orquestradores de containers, como o Amazon Elastic Container Service (ECS), oferecem esses mecanismos de forma declarativa: o operador informa quantas cópias de uma aplicação devem existir e em que condições esse número muda, e o orquestrador mantém o estado desejado.

O objetivo da equipe é projetar, implantar e avaliar uma aplicação HTTP containerizada no ECS, demonstrando, com medições, três propriedades:

- **replicação de serviço**: várias tasks atendem o mesmo serviço e cada resposta identifica a task que a produziu;
- **tolerância a falhas**: ao encerrar uma task, o ECS inicia uma substituta e o serviço volta ao número desejado;
- **escalabilidade automática**: o número de tasks aumenta sob carga e diminui quando ela cai.

Banco de dados, interface web e funcionalidades de negócio ficaram fora do escopo. A pilha escolhida foi FastAPI, Docker, Amazon ECR, Amazon ECS com Fargate, Application Load Balancer (ALB), CloudWatch e k6.

2. Fundamentação

2.1 Amazon ECS e Fargate

O Amazon ECS é um serviço gerenciado de orquestração de containers. Seus conceitos centrais são o **cluster** (agrupamento lógico), a **task definition** (especificação de imagem, CPU, memória, portas, logs e health check), a **task** (uma instância em execução da task definition) e o **service**, que mantém um número desejado de tasks (*desired count*) e, opcionalmente, as registra em um load balancer. Com o tipo de execução **Fargate**, a AWS provisiona a capacidade de computação de cada task, sem que a equipe administre instâncias EC2 [2].

2.2 Serviço replicado e balanceamento

Replicar um serviço sem estado (stateless) permite que qualquer réplica atenda qualquer requisição. Um balanceador fornece um endereço único, distribui as requisições e retira de circulação as réplicas que falham no health check. No projeto, o ALB cumpre esse papel; sem ele, cada task teria um IP próprio que muda a cada substituição, o que impede medir scale-out e recuperação de forma contínua.

2.3 Tolerância a falhas

O ECS service compara continuamente o número de tasks em execução com o desejado. Quando uma task para, seja por falha, por health check reprovado ou por encerramento manual, o agendador inicia outra. O tempo de recuperação observado depende do início do container, do período de carência e dos parâmetros dos health checks; por isso esses parâmetros fazem parte da configuração registrada em cada experimento.

2.4 Escalabilidade automática

O Application Auto Scaling ajusta o desired count de um ECS service. Na política de **target tracking**, define-se um valor-alvo para uma métrica, aqui a CPU média do serviço, e o próprio serviço cria alarmes no CloudWatch que disparam o aumento ou a redução de tasks para aproximar a métrica do alvo [3]. Os cooldowns limitam a frequência das mudanças. A redução é, em geral, mais conservadora que o aumento, para evitar oscilação.

2.5 Trade-offs

- **Custo × disponibilidade**: mais tasks e um ALB aumentam a disponibilidade e a capacidade, mas cobram por hora.
- **Reatividade × estabilidade**: alvo de CPU baixo e cooldown curto reagem mais rápido, mas podem oscilar.
- **Simplicidade × controle**: Fargate dispensa gerenciar servidores, mas limita os tamanhos de task e o acesso ao host.
- **ECS × EKS**: o ECS é integrado à AWS e mais simples de operar; o EKS (Kubernetes) oferece portabilidade e um ecossistema maior, ao custo de mais complexidade operacional [1].

3. Arquitetura e implementação

O Quadro 1 resume a arquitetura projetada e versionada no repositório (`docs/arquitetura-custos.md`, `infrastructure/templates/`).

Componente	Configuração
Gerador de carga	k6 1.0, modelo aberto (taxa de chegada fixa) contra GET /work
Entrada	ALB; health check /health a cada 10 s, 2 acertos ou 2 falhas; drenagem de 30 s
ECS service	Fargate, Linux x86_64, rede awsvpc, 1 a 3 tasks de 0,25 vCPU e 512 MiB
Rede	VPC existente, subnets públicas, IP público por task, sem NAT Gateway; tasks aceitam só o SG do ALB
Imagem	Amazon ECR, tag fixa por versão, task definition referencia o digest
Observabilidade	CloudWatch Logs (awslogs, retenção de 7 dias) e métricas AWS/ECS e AWS/ApplicationELB
Auto Scaling	Target tracking ECSServiceAverageCPUUtilization; alvo 60%; cooldown 60 s (out) e 180 s (in)

Quadro 1 — Componentes da arquitetura proposta. Fonte: arquivos citados no texto.

3.1 Aplicação (Pessoa 1)

A API foi escrita em Python 3.12 com FastAPI 0.115.6 e servida por Uvicorn 0.34.0 (`app/`). É stateless e expõe três rotas, todas com respostas JSON; toda resposta traz os headers `x-instance-id` e `x-request-id`.

Rota	Comportamento	Papel nos experimentos
GET /health	Retorna {"status":"ok"}; assíncrona, responde no event loop	Health check do container e do ALB
GET /instance	ID da task (metadados ECS v4), versão, AZ, PID, uptime	Distinguir réplicas

Rota	Comportamento	Papel nos experimentos
GET /work?iterations=N	N hashes SHA-256 encadeados; padrão 200 000, máximo 5 000 000; digest determinístico	Carga de CPU controlada

A imagem parte de `python:3.12-slim`, roda como usuário não-root, tem cerca de 55 MB e é construída para `linux/amd64`. A configuração é feita só por variáveis de ambiente, sem segredos. Os logs saem em JSON, uma linha por requisição, em stdout. Há 11 testes automatizados (pytest) e um workflow de CI que roda os testes e o build.

3.2 Infraestrutura (Pessoa 2)

Toda a infraestrutura é descrita por scripts PowerShell 7 e modelos JSON versionados (`infrastructure/`), usando a AWS CLI v2. O ciclo cobre verificação do ambiente (`Test-Environment`), provisionamento de ECR, cluster, security groups e ALB (`Provision`), publicação da imagem (`Publish-Image`), implantação (`Deploy`), consulta, teste HTTP e limpeza (`Remove-Infrastructure`).

- Os scripts recusam operar sem conta conferida, orçamento e janela registrados.
- Cada recurso recebe tags de propriedade e entra num inventário; a limpeza só remove o que comprovadamente foi criado.
- Não são criados roles IAM, VPC, sub-redes nem NAT Gateway; credenciais não são versionadas.
- 35 casos de teste offline, executados no GitHub Actions sem acesso à AWS.

3.3 Auto Scaling e experimentos (Pessoa 3)

Os scripts `Set-AutoScaling.ps1` e `Remove-AutoScaling.ps1` registram o serviço no Application Auto Scaling e criam ou removem a política de target tracking. As ferramentas de medição ficam em `tests/`: o script k6 com os cenários, o orquestrador `Invoke-Experiment.ps1` (pré-condições, k6, amostragem do ECS e do ALB a cada 10 s, `stop-task` em D, exportação de métricas, eventos e alarmes) e a análise `analisar.py`, que gera resumos, gráficos e a tabela experimental.

4. Decisão técnica: o impedimento API-02

Na integração, a equipe verificou que, com 0,25 vCPU, vários cálculos de `/work` em paralelo disputavam a GIL do Python e atrasavam o `/health` além do timeout. Sob carga, o ECS consideraria a task doente e a substituiria, o que invalidaria os experimentos de escalabilidade. A Tabela 1 resume as medições locais antes e depois da correção.

Situação (docker --cpus 0.25)	Falhas de /health (timeout 5 s)	/health p50 / p95	Vazão de /work
Antes: 20 clientes, 60 s	85%	—	—
Depois: 20 clientes, 90 s	0 de 90	86 / 186 ms	1,81 req/s
Depois: 60 clientes, 60 s	0 de 60	78 / 173 ms	1,80 req/s

Tabela 1 — Efeito da correção do API-02. Fonte: docs/validacao-local.md (12/11/2026).

A correção tornou `/health` assíncrono e passou a executar `/work` numa thread, com no máximo `WORK_CONCURRENCY` (padrão 1) cálculos simultâneos; os demais pedidos esperam na fila sem

consumir CPU. A vazão por task permanece em cerca de 1,8 req/s, e a latência de `/work` passa a crescer com a fila, que é exatamente o efeito que o scale-out deve reduzir. O health check da task também foi trocado por um comando com `socket`, que custa cerca de 0,10 s de CPU por execução, contra 0,30 s do anterior.

5. Metodologia

Como `/work` é limitado por CPU e cada task processa um cálculo por vez, a unidade de carga é **C**, a vazão máxima de uma task, medida no cenário de calibração no mesmo dia dos experimentos. Todas as cargas são frações de C, o que torna o plano independente do hardware. Localmente, C ≈ 1,86 req/s; no Fargate, C ainda precisa ser medido. Com carga L e N tasks, a CPU média é aproximadamente L / (N·C), e o target tracking busca N ≈ L / (0,6·C).

Experimento	Tasks	Carga	Hipótese
A · Baseline	1, sem Auto Scaling	0,5 C por 5 min	Vazão ≥ 95% da oferecida, erros < 1%, CPU entre 35% e 65%
B · Scale-out	1 → 3	0,3 C; degraus de 0,85 C (6 min) e 1,5 C (7 min)	Desejadas 1→2→3 e p95 menor com mais tasks na mesma carga
C · Scale-in	3 → 1	0,2 C por 22 min + 5 min	Retorno a 1 task, erros < 1%
D · Falha	2, sem Auto Scaling	0,8 C por 10 min; stop-task aos 3 min	Substituta saudável no ALB; erros na janela < 1%

Tabela 2 — Desenho experimental. Fonte: tests/plano-de-testes.md.

Os degraus de B ficam abaixo da capacidade das tasks existentes: com mais carga do que a capacidade, a fila cresceria sem limite e o experimento passaria a medir a fila, não o Auto Scaling. Os tempos são definidos assim:

- **Scale-out**: do alarme de alta entrar em ALARM até o ALB ter alvos saudáveis em número igual às novas tasks desejadas;
- **Scale-in**: do início da redução de carga até a redução das tasks desejadas;
- **Recuperação**: do pedido de stop-task até as tasks saudáveis voltarem ao número anterior.

Indicadores coletados: vazão, latência p50/p95/p99, erros e distribuição por task (k6); tasks desejadas, em execução e saudáveis (ECS e ALB, a cada 10 s); CPU e memória (CloudWatch, períodos de 1 min); histórico de alarmes, atividades do Auto Scaling e eventos do ECS. A amostragem de 10 s e as métricas de 1 min limitam a precisão dos tempos. Estão previstas 3 repetições por cenário, se orçamento e prazo permitirem.

6. Resultados disponíveis: validação local das ferramentas

Atenção. Os resultados abaixo vêm do ambiente `tests/local`: 3 réplicas Docker com 0,25 vCPU e 512 MiB atrás de um nginx, **sem ECS, ALB nem Auto Scaling**, com tempos comprimidos e uma repetição por cenário. Eles validam o k6, o orquestrador e a análise; não são evidência dos experimentos na AWS.

Execução (13/11/2026)	Vazão (req/s)	p95 (ms)	Erros	Observação
Smoke	2,26	500,0	0%	—
Calibração	1,74	3 890,6	0%	Fila sempre cheia, por desenho
A · Baseline	0,85	330,6	0%	1 réplica
B/C · Escalabilidade	1,02	398,7	0%	3 réplicas fixas
D · Falha	1,38	479,2	0,8%	Recuperação em 35 s (20 s simulados)

Tabela 3 — Execuções de validação local. Fonte: tests/results/exemplo-local/tabela-experimental.md (branch pessoa3/experimentos).

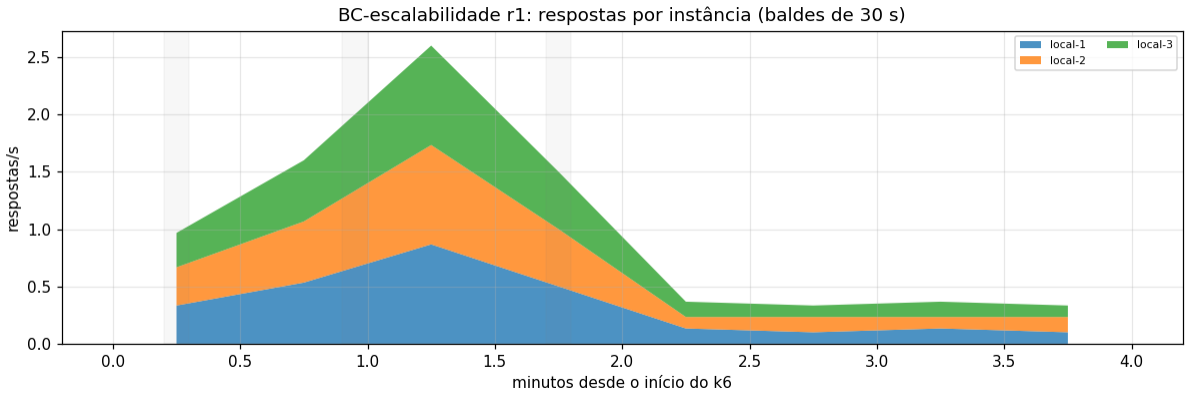


Figura 1 — Respostas por réplica no cenário BC-escalabilidade local (baldes de 30 s): as três réplicas fixas dividem a carga. Fonte: tests/results/exemplo-local/20261113-021744-BC-escalabilidade-r1-local/grafico-instancias.png.

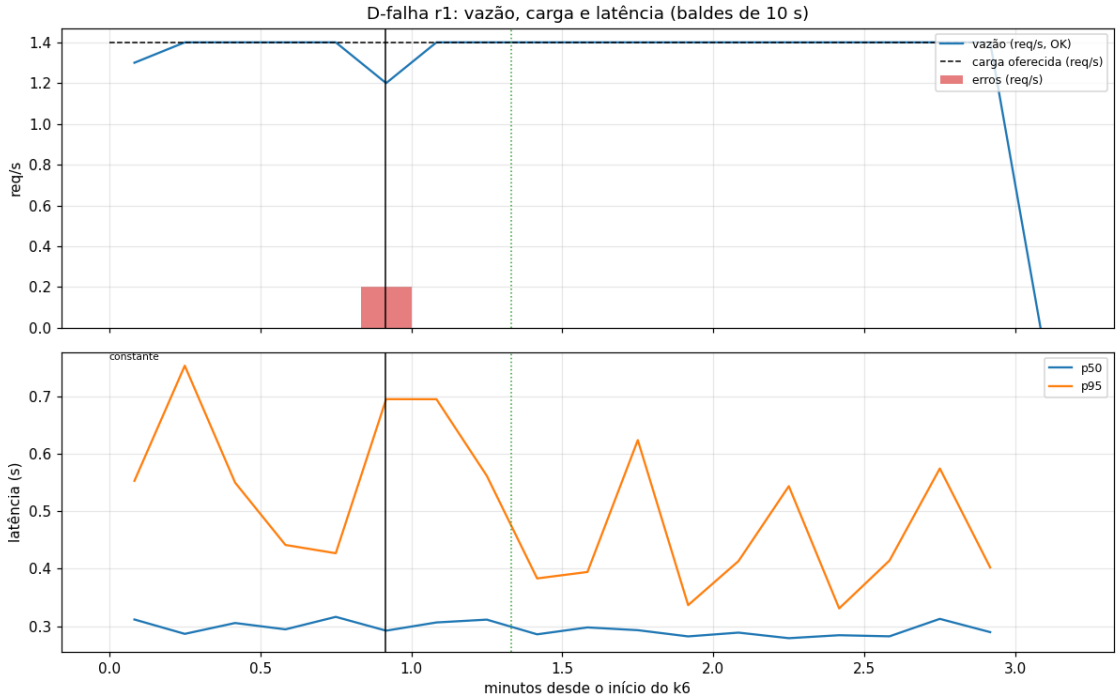


Figura 2 — Cenário D-falha local: vazão, carga oferecida, erros e latência (baldes de 10 s). A linha preta marca a parada da réplica e a verde, a recuperação. Fonte: tests/results/exemplo-local/20261113-022236-D-falha-r1-local/grafico-carga-latencia.png.

No cenário D local, houve 2 erros em 98 requisições na janela da falha, e o p95 foi de 549,9 ms antes, 479,2 ms durante e 414,2 ms depois da parada. Dos 35 s de recuperação, 20 s são o atraso de reposição simulado pelo script, pois não há ECS no ambiente local.

7. Discussão e limitações

- **Sem dados da nuvem.** Até 14/11, os impedimentos AWS-01 (conta), CUSTO-01 (orçamento e janela) e EXP-01 (ALB e Auto Scaling autorizados) estavam abertos. Nenhum tempo de scale-out, scale-in ou recuperação no ECS foi medido.
- **Mais tasks não provam desempenho.** As conclusões precisam relacionar o número de tasks à vazão, à latência e aos erros sob a mesma carga oferecida; a análise já separa as requisições pelo número de tasks saudáveis.
- **Fila sem limite na API.** Acima da capacidade, os pedidos esperam e os que passam de 60 s recebem 504 do ALB, mas continuam sendo processados. O desenho em degraus evita a situação; descartar excedentes seria uma melhoria futura.
- **Gerador fora da AWS.** A latência medida pelo k6 inclui a internet; o local de origem deve ser mantido entre repetições.
- **Variação do Fargate.** A CPU de 0,25 vCPU pode variar entre tasks e horários; por isso a calibração ocorre no mesmo dia.
- **Parâmetros dos health checks** (10 s, 2 acertos, drenagem de 30 s) influenciam os tempos e fazem parte da configuração.

8. Custos e limpeza

Nenhum valor foi registrado porque região, créditos, janela e limite ainda não foram definidos. A estimativa deve ser feita na AWS Pricing Calculator considerando os componentes abaixo; o roteiro completo dos experimentos usa cerca de 3 horas de ALB com 1 a 3 tasks.

Componente	Base de cobrança a considerar
Fargate	Horas de task × 0,25 vCPU e 0,5 GiB, incluindo inicialização e sobreposição de deploy
IPv4 público	IP-horas das tasks e do ALB
ALB	Horas do balanceador e unidades de capacidade
ECR	Armazenamento das imagens e transferências
CloudWatch	Ingestão e retenção de logs, métricas e alarmes

Tabela 4 — Componentes de custo. Fonte: docs/arquitetura-custos.md.

Ao fim de cada sessão, os dados e as evidências são salvos e os recursos removidos com `Remove-AutoScaling.ps1` e `Remove-Infrastructure.ps1`. Reduzir o desired count a zero não remove ALB, repositório nem logs.

9. Contribuições da equipe

A divisão segue o plano da equipe e foi conferida no histórico Git de todas as branches. Os commits `cbc0b12` e `865b916` estão na branch `pessoa3/experimentos`, ainda sem merge em `main`.

Integrante	Responsabilidade	Commits	Entregas
Pessoa 1 (LandriAlencars)	Aplicação, Docker e imagem	f8458a6	API, Dockerfile, 8 testes, evidência de 2 instâncias, CI da app
Pessoa 2 (PabloMuh)	Infraestrutura AWS e implantação	551ed20, 7d825ab	Scripts, task definition, service, checklist, custos, impedimentos
Pessoa 3 (leonardo-lbp)	Auto Scaling, carga e testes	f2d6cf3, d426ecd, cbc0b12, 865b916	Health check leve, correção API-02, Auto Scaling, k6, orquestrador, análise
Pessoa 4	Observabilidade e documentação	pessoa4/documentacao	docs/PROJETO.md, este relatório, apresentação

Tabela 5 — Contribuições por integrante. Fonte: git log --all.

10. Próximos passos

- Até 19/11 (marco M4): validar conta e orçamento, provisionar com ALB, publicar a imagem com tag fixa e registrar o digest.
- Calibrar C no Fargate e executar A, B/C e D, com 3 repetições se possível.
- Validar com a Pessoa 2 a captura de logs e métricas no CloudWatch e definir o padrão dos registros.
- 19 a 23/11: consolidar os dados, produzir os gráficos finais com unidades e metodologia e discutir os resultados.
- Registrar o custo real da sessão e executar a limpeza dos recursos.

Referências

[1] YERRAM, S. *AWS EKS vs AWS ECS: A Research-Based Comparison for Container Orchestration*. Zenodo, 2025. Disponível em: https://zenodo.org/records/16500754.

[2] AMAZON WEB SERVICES. *What is Amazon Elastic Container Service?* Disponível em: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html.

[3] AMAZON WEB SERVICES. *Automatically scale your Amazon ECS service*. Disponível em: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-auto-scaling.html.